DIGITAL ELECTRONICS ENGINEERING LAB

50 Digital Logic Circuits

Functions and Truth Tables

A compact reference for the 50 circuits listed in the supplied DEE lab index. Each entry gives the Boolean function and a truth table, state table, or standard selector mapping appropriate to that circuit.

Table conventions: apostrophe means NOT; + means OR; multiplication or adjacency means AND; XOR means exclusive OR; X means invalid or don't-care as stated.

The lab PDF contains topic names only. For the Boolean-expression topics, representative functions are used and identified in the entries: F(A,B,C)=A'B+BC'+AC; F(A,B,C,D)=A'B+CD'+AB'D; and F(A,B)=A'B+AB'.

Reference notes

Encoders assume one active input at a time. Decoder outputs are active-high. The seven-segment circuit is common-cathode with active-high segment outputs. The SR circuits use the active-high or active-low inputs shown in their tables. The D, JK, and T master-slave circuits are falling-edge triggered; the gated SR circuits are level-sensitive.

Circuit index

01 1-to-4 demultiplexer
02 2-to-4 decoder
03 3-to-8 decoder
04 4-to-1 multiplexer
05 8-to-1 multiplexer
06 Octal-to-binary encoder
07 Decimal-to-BCD encoder
08 2-to-1 multiplexer
09 3-bit even parity generator using NAND gates
10 4-bit even parity generator using NAND gates
11 AND gate using NAND gate
12 AND gate using NOR gate
13 D flip-flop using NAND gates
14 Excess-3 code conversion from BCD
15 Full adder
16 Full subtractor
17 Half adder
18 Half subtractor
19 Three-variable Boolean expression using basic gates
20 Three-variable Boolean expression using NAND gates
21 JK flip-flop using NAND gates
22 Hexadecimal-to-binary encoder
23 NAND gate using NOR gates
24 NOR gate using NAND gates
25 OR gate using NAND gates
26 OR gate using NOR gates
27 SR flip-flop using NAND gates
28 SR flip-flop using NOR gates
29 SR latch using NAND gates
30 SR latch using NOR gates
31 3-bit seven-segment display
32 XNOR gate using NAND gates
33 XNOR gate using NOR gates
34 XOR gate using NAND gates
35 XOR gate using NOR gates
36 AND gate
37 OR gate
38 NOT gate
39 XOR gate
40 XNOR gate
41 NOR gate
42 NAND gate
43 T flip-flop using NAND gates
44 Three-variable Boolean expression using NOR gates
45 Four-variable Boolean expression using basic gates
46 Four-variable Boolean expression using NAND gates
47 NOT gate using NAND gate
48 NOT gate using NOR gate
49 Two-variable Boolean expression using basic gates
50 Two-variable Boolean expression using NAND gates

Notation note: for muxes and encoders, the table lists selected-input or one-hot mappings instead of expanding every possible combination of data inputs.

01 1-to-4 demultiplexer

Function: Y0=D*S1'*S0'; Y1=D*S1'*S0; Y2=D*S1*S0'; Y3=D*S1*S0.

D	S1	S0	Y0	Y1	Y2	Y3
0	0	0	0	0	0	0
0	0	1	0	0	0	0
0	1	0	0	0	0	0
0	1	1	0	0	0	0
1	0	0	1	0	0	0
1	0	1	0	1	0	0
1	1	0	0	0	1	0
1	1	1	0	0	0	1

Note: *S1 is the most-significant select bit.*

02 2-to-4 decoder

Function: Y0=A'*B'; Y1=A'*B; Y2=A*B'; Y3=A*B.

A	B	Y0	Y1	Y2	Y3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

Note: *Active-high, one-hot output.*

03 3-to-8 decoder

Function: Yi=1 only when ABC is the 3-bit binary value i, for i=0..7.

A	B	C	Y0	Y1	Y2	Y3	Y4	Y5	Y6	Y7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

Note: *A is the most-significant input bit; outputs are active-high.*

04 4-to-1 multiplexer

Function: Y=D0*S1'*S0' + D1*S1'*S0 + D2*S1*S0' + D3*S1*S0.

S1	S0	Y
0	0	D0
0	1	D1
1	0	D2
1	1	D3

Note: *The table shows the selected data input; Y equals that input.*

05 8-to-1 multiplexer

Function: Y=OR(Di*mi(S2,S1,S0)) for i=0..7; mi is the minterm whose binary value is i.

S2	S1	S0	Y
0	0	0	D0
0	0	1	D1
0	1	0	D2
0	1	1	D3
1	0	0	D4
1	0	1	D5
1	1	0	D6
1	1	1	D7

Note: *The table shows the selected data input; Y equals that input.*

06 Octal-to-binary encoder

Function: B2=D4+D5+D6+D7; B1=D2+D3+D6+D7; B0=D1+D3+D5+D7.

Active input	B2	B1	B0
D0	0	0	0
D1	0	0	1
D2	0	1	0
D3	0	1	1
D4	1	0	0
D5	1	0	1
D6	1	1	0
D7	1	1	1

Note: *One-hot input: exactly one Di is 1.*

07 Decimal-to-BCD encoder

Function: B3=D8+D9; B2=D4+D5+D6+D7; B1=D2+D3+D6+D7; B0=D1+D3+D5+D7+D9.

Active input	B3	B2	B1	B0
D0	0	0	0	0
D1	0	0	0	1
D2	0	0	1	0
D3	0	0	1	1
D4	0	1	0	0
D5	0	1	0	1
D6	0	1	1	0
D7	0	1	1	1
D8	1	0	0	0
D9	1	0	0	1

Note: *One-hot input: exactly one decimal input is 1.*

08 2-to-1 multiplexer

Function: Y=D0*S' + D1*S.

S	Y
0	D0
1	D1

Note: *The table shows the selected data input; Y equals that input.*

09 3-bit even parity generator using NAND gates

Function: P=A XOR B XOR C.

A	B	C	P
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

Note: *Append P to A,B,C; the total number of 1s is then even.*

10 4-bit even parity generator using NAND gates

Function: P=A XOR B XOR C XOR D.

A	B	C	D	P
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0

Note: *Append P to A,B,C,D; the total number of 1s is then even.*

11 AND gate using NAND gate

Function: Y=A*B. Realization: T=NAND(A,B); Y=NAND(T,T).

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

12 AND gate using NOR gate

Function: Y=A*B. Realization: A'=NOR(A,A); B'=NOR(B,B); Y=NOR(A',B').

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

13 D flip-flop using NAND gates

Function: At a falling clock edge, Q+=D and Qbar+=D'. Between edges, Q+=Q.

Clock event	D	Q	Q+	Qbar+
Falling edge	0	0	0	1
Falling edge	0	1	0	1
Falling edge	1	0	1	0
Falling edge	1	1	1	0
No edge	X	0	0	1
No edge	X	1	1	0

Note: *Master-slave implementation; falling-edge triggered.*

14 Excess-3 code conversion from BCD

Function: For valid BCD A B C D, add 0011. E0=D'; E1=C XNOR D; E2=B XOR (C+D); E3=A XOR [B*(C+D)].

Decimal	BCD A B C D	Excess-3 E3 E2 E1 E0
0	0 0 0 0	0 0 1 1
1	0 0 0 1	0 1 0 0
2	0 0 1 0	0 1 0 1
3	0 0 1 1	0 1 1 0
4	0 1 0 0	0 1 1 1
5	0 1 0 1	1 0 0 0
6	0 1 1 0	1 0 0 1
7	0 1 1 1	1 0 1 0
8	1 0 0 0	1 0 1 1
9	1 0 0 1	1 1 0 0

Note: *Inputs 1010 through 1111 are invalid BCD and are not listed.*

15 Full adder

Function: S=A XOR B XOR Cin; Cout=A*B + Cin*(A XOR B).

A	B	Cin	S	Cout
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

16 Full subtractor

Function: Diff=A XOR B XOR Bin; Bout=A'*B + (A XNOR B)*Bin.

A	B	Bin	Diff	Bout
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

17 Half adder

Function: S=A XOR B; Carry=A*B.

A	B	S	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

18 Half subtractor

Function: Diff=A XOR B; Borrow=A'*B.

A	B	Diff	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

19 Three-variable Boolean expression using basic gates

Function: F=A'*B + B*C' + A*C.

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Note: *Representative function: the supplied PDF index does not specify an expression.*

20 Three-variable Boolean expression using NAND gates

Function: F=A'*B + B*C' + A*C (NAND-only realization).

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Note: *Representative function: the supplied PDF index does not specify an expression.*

21 JK flip-flop using NAND gates

Function: At a falling edge, Q+=J*Q' + K'*Q.

J	K	Q	Q+
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Note: *J=K=1 toggles; J=K=0 holds.*

22 Hexadecimal-to-binary encoder

Function: B3=D8+D9+DA+DB+DC+DD+DE+DF; B2=D4+D5+D6+D7+DC+DD+DE+DF; B1=D2+D3+D6+D7+DA+DB+DE+DF; B0=D1+D3+D5+D7+D9+DB+DD+DF.

Active input	B3	B2	B1	B0
D0	0	0	0	0
D1	0	0	0	1
D2	0	0	1	0
D3	0	0	1	1
D4	0	1	0	0
D5	0	1	0	1
D6	0	1	1	0
D7	0	1	1	1
D8	1	0	0	0
D9	1	0	0	1
DA	1	0	1	0
DB	1	0	1	1
DC	1	1	0	0
DD	1	1	0	1
DE	1	1	1	0
DF	1	1	1	1

Note: *One-hot input: exactly one hexadecimal input is 1.*

23 NAND gate using NOR gates

Function: Y=(A*B)'. NOR-only realization: invert A and B with self-NOR, NOR them to get A*B, then invert.

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

24 NOR gate using NAND gates

Function: Y=(A+B)'. NAND-only realization: make A+B with NAND inverters, then invert with a self-NAND.

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

25 OR gate using NAND gates

Function: Y=A+B. Realization: NAND(A',B') with A'=NAND(A,A), B'=NAND(B,B).

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

26 OR gate using NOR gates

Function: Y=A+B. Realization: T=NOR(A,B); Y=NOR(T,T).

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

27 SR flip-flop using NAND gates

Function: Sbar_int=NAND(S,EN); Rbar_int=NAND(R,EN); cross-coupled NAND latch.

EN	S	R	Q	Q+
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	0
1	1	0	0	1
1	1	0	1	1
1	1	1	0	X
1	1	1	1	X

Note: *Level-sensitive enable. EN=0 holds; with EN=1, 00 holds, 10 sets, 01 resets, and 11 is invalid.*

28 SR flip-flop using NOR gates

Function: Sg=S*EN; Rg=R*EN; Q=NOR(Rg,Qbar); Qbar=NOR(Sg,Q).

EN	S	R	Q	Q+
0	0	0	0	0
0	0	0	1	1
0	0	1	0	0
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	0
1	0	1	1	0
1	1	0	0	1
1	1	0	1	1
1	1	1	0	X
1	1	1	1	X

Note: *Level-sensitive enable. EN=0 holds; with EN=1, 00 holds, 10 sets, 01 resets, and 11 is invalid.*

29 SR latch using NAND gates

Function: Q=NAND(Sbar,Qbar); Qbar=NAND(Rbar,Q).

Sbar	Rbar	Q	Q+	Qbar+
0	0	0	X	X
0	0	1	X	X
0	1	0	1	0
0	1	1	1	0
1	0	0	0	1
1	0	1	0	1
1	1	0	0	1
1	1	1	1	0

Note: *Inputs are active-low. Sbar=Rbar=0 is invalid.*

30 SR latch using NOR gates

Function: Q=NOR(R,Qbar); Qbar=NOR(S,Q).

S	R	Q	Q+	Qbar+
0	0	0	0	1
0	0	1	1	0
0	1	0	0	1
0	1	1	0	1
1	0	0	1	0
1	0	1	1	0
1	1	0	X	X
1	1	1	X	X

Note: *Inputs are active-high. S=R=1 is invalid.*

31 3-bit seven-segment display

Function: a=1 for digits 0,2,3,5,6,7; b=1 for digits 0,1,2,3,4,7; c=1 for digits 0,1,3,4,5,6,7; d=1 for digits 0,2,3,5,6; e=1 for digits 0,2,6; f=1 for digits 0,4,5,6; g=1 for digits 2,3,4,5,6.

A	B	C	a	b	c	d	e	f	g
0	0	0	1	1	1	1	1	1	0
0	0	1	0	1	1	0	0	0	0
0	1	0	1	1	0	1	1	0	1
0	1	1	1	1	1	1	0	0	1
1	0	0	0	1	1	0	0	1	1
1	0	1	1	0	1	1	0	1	1
1	1	0	1	0	1	1	1	1	1
1	1	1	1	1	1	0	0	0	0

Note: *Common-cathode display; segment outputs are active-high. A is the most-significant input.*

32 XNOR gate using NAND gates

Function: Y=(A XOR B)'. Implement XOR with four NAND gates and invert the result with a fifth NAND.

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

33 XNOR gate using NOR gates

Function: Y=(A XOR B)'. Four-NOR XNOR network.

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

34 XOR gate using NAND gates

Function: Y=A XOR B. Standard four-NAND network.

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

35 XOR gate using NOR gates

Function: Y=A XOR B. NOR-only realization.

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

36 AND gate

Function: Y=A*B.

A	B	Y
0	0	0
0	1	0
1	0	0
1	1	1

37 OR gate

Function: Y=A+B.

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	1

38 NOT gate

Function: Y=A'.

A	Y
0	1
1	0

39 XOR gate

Function: Y=A XOR B.

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0

40 XNOR gate

Function: Y=(A XOR B)'.

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	1

41 NOR gate

Function: Y=(A+B)'.

A	B	Y
0	0	1
0	1	0
1	0	0
1	1	0

42 NAND gate

Function: Y=(A*B)'.

A	B	Y
0	0	1
0	1	1
1	0	1
1	1	0

43 T flip-flop using NAND gates

Function: At a falling edge, Q+=T XOR Q.

T	Q	Q+
0	0	0
0	1	1
1	0	1
1	1	0

Note: *T=0 holds; T=1 toggles.*

44 Three-variable Boolean expression using NOR gates

Function: F=A'*B + B*C' + A*C (NOR-only realization).

A	B	C	F
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Note: *Representative function: the supplied PDF index does not specify an expression.*

45 Four-variable Boolean expression using basic gates

Function: F=A'*B + C*D' + A*B'*D.

A	B	C	D	F
0	0	0	0	0
0	0	0	1	0
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	1
1	1	1	1	0

Note: *Representative function: the supplied PDF index does not specify an expression.*

46 Four-variable Boolean expression using NAND gates

Function: F=A'*B + C*D' + A*B'*D (NAND-only realization).

A	B	C	D	F
0	0	0	0	0
0	0	0	1	0
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	1
0	1	1	0	1
0	1	1	1	1
1	0	0	0	0
1	0	0	1	1
1	0	1	0	1
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	1
1	1	1	1	0

Note: *Representative function: the supplied PDF index does not specify an expression.*

47 NOT gate using NAND gate

Function: Y=NAND(A,A)=A'.

A	Y
0	1
1	0

48 NOT gate using NOR gate

Function: Y=NOR(A,A)=A'.

A	Y
0	1
1	0

49 Two-variable Boolean expression using basic gates

Function: F=A'*B + A*B' = A XOR B.

A	B	F
0	0	0
0	1	1
1	0	1
1	1	0

Note: *Representative function: the supplied PDF index does not specify an expression.*

50 Two-variable Boolean expression using NAND gates

Function: F=A'*B + A*B' = A XOR B (NAND-only realization).

A	B	F
0	0	0
0	1	1
1	0	1
1	1	0

Note: *Representative function: the supplied PDF index does not specify an expression.*